

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет
ИТМО»

Мегафакультет компьютерных технологий и управления

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3
по предмету “базы данных”

Выполнила: Кобленц Мария Алексеевна

Группа: Р3106

Проверил: Мартин Райла

г. Санкт-Петербург

2026

1.....	4
1.1. Процесс проектирования и обоснование получения схемы.....	4
1.2. Перечень отношений и их атрибутов.....	4
Справочные таблицы (категории, параметры).....	4
Таблица Difficulty.....	4
Таблица Urgency.....	5
Таблица Priority.....	5
Таблица Frequency.....	5
Таблица Category.....	5
Таблица Role.....	5
Таблица GrowthStage.....	6
Таблица SeedState.....	6
Таблица PlanetSize.....	6
Таблица IntegrityStatus.....	6
Таблица Outcome.....	6
Таблица ConfusionRisk.....	7
Таблица RiskLevel.....	7
Таблица DiligenceLevel.....	7
Основные сущности предметной области.....	7
Таблица Rule.....	7
Таблица Planet.....	7
Таблица Action.....	8
Таблица Seed.....	8
Таблица Character.....	8
Таблица Plant.....	8
Журналы и аналитика.....	9
Таблица TaskLog.....	9
Таблица RiskCase.....	9
Таблица SpeciesSimilarity.....	9
Таблица ThreatLevel.....	9
2. Неудачное проектное решение.....	10
2.1. Описание неудачных отношений.....	10
2.2. Ошибки проектирования.....	11
Таблица Planet_Operations.....	11
Таблица Plant_Catalog.....	11
Таблица Character_Profile.....	12
2.3. Демонстрация аномалий на примерах SQL-операций.....	12
Аномалия 1.....	12
Аномалия 2.....	12
Аномалия 3.....	13
Аномалия 4.....	13
3. Определение функциональных зависимостей.....	14
Таблица Difficulty.....	14

Таблица Urgency.....	14
Таблица Category.....	14
Таблица GrowthStage.....	15
Таблица Planet.....	15
Таблица Seed.....	16
Таблица Plant.....	16
Таблица Character.....	16
Таблица TaskLog.....	17
Таблица SpeciesSimilarity.....	17
Таблица ThreatLevel.....	18
Таблица Priority.....	18
Таблица Frequency.....	19
Таблица Role.....	19
Таблица SeedState.....	19
Таблица PlanetSize.....	20
Таблица IntegrityStatus.....	20
Таблица Outcome.....	20
Таблица ConfusionRisk.....	21
Таблица RiskLevel.....	21
Таблица DiligenceLevel.....	21
Таблица Rule.....	21
Таблица Action.....	22
Таблица RiskCase.....	22
Сводное заключение по пункту 3.....	23
4. Приведение отношений к третьей нормальной форме (3NF).....	24
4.1. Доказательство соответствия 3NF для справочных отношений.....	24
4.2. Доказательство соответствия 3NF для основных бизнес-отношений.....	25
4.3. Доказательство соответствия 3NF для аналитических отношений.....	25
4.4. Преобразование отношения TaskLog (устранение нарушения 3NF).....	26
4.5. Итоговый вывод по пункту 4.....	27
5. Изменения в функциональных зависимостях после приведения к 3NF.....	28
5.1. Локализация функциональных зависимостей в отдельных отношениях.....	28
5.2. Зависимости, переставшие вызывать нарушения нормальных форм.....	28
5.3. Изменение набора зависимостей в полученной схеме.....	29
5.4. Сохранение зависимостей после декомпозиции.....	29
6. Проверка схемы на соответствие BCNF.....	30
Справочные отношения.....	30
Основные бизнес-отношения.....	31
Таблица Planet.....	31
Таблица Plant.....	31
Таблица Character.....	31
Журналы и аналитические отношения.....	32
Таблица TaskLog (после нормализации).....	32
Таблица ThreatLevel.....	32

Таблица SpeciesSimilarity.....	32
Итоговый вывод по пункту 6.....	33
7. Сравнение исходной схемы, схем после приведения к 3NF и BCNF, а также предложенные варианты денормализации.....	34
7.1. Сопоставление исходной схемы и нормализованных версий.....	34
7.2. Предлагаемые варианты денормализации.....	34
Вариант 1.....	34
Вариант 2.....	34
8. Разработка и реализация триггера на языке PL/pgSQL для системы управления задачами.....	35
8.1. Обоснование выбора триггера и бизнес-правила.....	35
8.2. Описание затрагиваемых таблиц и их назначения.....	36
Таблица Employee.....	36
Таблица Project.....	37
Таблица Task.....	37
Таблица Employee_Project_Rights.....	37
Таблица Notification.....	37
Таблица Task_Audit.....	37
Таблица Budget_Reservation.....	37
8.3. Код создания схемы базы данных.....	38
8.4. Реализация триггерной функции.....	40
8.5. Создание триггера.....	42
8.6. Примеры демонстрации работы триггера.....	42

1.

1.1. Процесс проектирования и обоснование получения схемы

Исходная схема отношений сформирована на основе последовательного анализа предметной области «Уход за планетой в мире Маленького принца». Процесс проектирования включал следующие этапы:

1. Выделение сущностей из текста предметной области.
На основе описания бизнес-правил («семена невидимы», «баобабы нужно вырывать вовремя», «планета может быть заражена», «персонаж выполняет действия») были идентифицированы ключевые объекты: Планета, Растение, Семя, Персонаж, Действие, Правило, Стадия роста, Категория растений, Уровень угрозы и другие.
2. Определение атрибутов и связей.
Для каждой сущности выделены свойства, необходимые для описания ее состояния и поведения в рамках предметной области. Например, для растения важны стадия роста, степень узнаваемости, уровень проникновения корней и принадлежность к категории. Связи установлены по принципу «один-ко-многим» и «многие-ко-многим» с учетом логики предметной области (одно растение растет на одной планете, один персонаж выполняет много действий, одно действие может применяться к разным растениям).
3. Формирование первоначальной структуры.
Интуитивно казалось, что для удобства редактирования данных каждая сущность должна лежать в отдельной таблице, а составные сущности содержать ссылки на части. Первоначальная структура формировалась с опорой на разделение ответственности.

1.2. Перечень отношений и их атрибутов

Ниже представлена полная спецификация схемы. Все первичные ключи — суррогатные (SERIAL PRIMARY KEY).

Справочные таблицы (категории, параметры)

Таблица Difficulty

Атрибуты: difficulty_id, difficulty_name, success_rate_modifier.

Первичный ключ: difficulty_id.

Семантический кандидатный ключ: difficulty_name.

Назначение: классификация сложности действий.

Бизнес-смысл: «Вырвать баобаб» — сложнее, чем «полить розу»; влияет на вероятность успеха.

Таблица Urgency

Атрибуты: urgency_id, urgency_name, max_delay_hours.

Первичный ключ: urgency_id.

Семантический кандидатный ключ: urgency_name.

Назначение: уровень срочности действий.

Бизнес-смысл: баобабы требуют немедленного вмешательства; розы могут подождать.

Таблица Priority

Атрибуты: priority_id, priority_name.

Первичный ключ: priority_id.

Семантический кандидатный ключ: priority_name.

Назначение: приоритет правил.

Бизнес-смысл: определяет, какое правило важнее при конфликте.

Таблица Frequency

Атрибуты: frequency_id, frequency_name, interval_days.

Первичный ключ: frequency_id.

Семантический кандидатный ключ: frequency_name.

Назначение: периодичность выполнения правил.

Бизнес-смысл: «Каждый день выпалывать баобабы» — частота interval_days = 1.

Таблица Category

Атрибуты: category_id, category_name, is_dangerous.

Первичный ключ: category_id.

Семантический кандидатный ключ: category_name.

Назначение: тип растения: полезное или вредное
Бизнес-смысл: is_dangerous = TRUE → баобабы, сорняки; FALSE → розы, редис.

Таблица Role

Атрибуты: role_id, role_name, is_available, can_delete_harmful_plants.

Первичный ключ: role_id.

Семантический кандидатный ключ: role_name.

Назначение: роли персонажей.

Бизнес-смысл: садовник может удалять вредные растения; наблюдатель — нет.

Таблица GrowthStage

Атрибуты: stage_id, stage_name, stage_order.

Первичный ключ: stage_id.

Семантический кандидатный ключ: stage_name или stage_order.

Назначение: стадии роста растения.

Бизнес-смысл: молодой росток баобаба похож на розу; на поздней стадии — уже поздно.

Таблица SeedState

Атрибуты: state_id, state_name, is_removable.

Первичный ключ: state_id.

Семантический кандидатный ключ: state_name.

Назначение: состояние семени.

Бизнес-смысл: is_removable = TRUE → семя можно удалить до прорастания.

Таблица PlanetSize

Атрибуты: size_id, size_name, description, max_safe_plants.

Первичный ключ: size_id.

Семантический кандидатный ключ: size_name.

Назначение: размер планеты.

Бизнес-смысл: маленькая планета → меньше допустимое количество баобабов (max_safe_plants).

Таблица IntegrityStatus

Атрибуты: status_id, status_name, requires_immediate_action.

Первичный ключ: status_id.

Семантический кандидатный ключ: status_name.

Назначение: состояние целостности планеты.

Бизнес-смысл: «Заражена», «Требуется немедленных действий» — триггеры для срочных мер.

Таблица Outcome

Атрибуты: outcome_id, outcome_name, description.

Первичный ключ: outcome_id.

Семантический кандидатный ключ: outcome_name.

Назначение: исход рискованного события.

Бизнес-смысл: «Планета уничтожена», «Баобаб удален вовремя».

Таблица ConfusionRisk

Атрибуты: risk_id, risk_name.

Первичный ключ: risk_id.

Семантический кандидатный ключ: risk_name.

Назначение: риск спутать виды.

Бизнес-смысл: «Баобаб и роза на ранней стадии» — высокий риск ошибки.

Таблица RiskLevel

Атрибуты: risk_level_id, risk_level_name.

Первичный ключ: risk_level_id.

Семантический кандидатный ключ: risk_level_name.

Назначение: уровень риска.

Бизнес-смысл: «Низкий», «Критический» — для визуализации и приоритезации.

Таблица DiligenceLevel

Атрибуты: level_id, level_name, avg_task_delay_days.

Первичный ключ: level_id.

Семантический кандидатный ключ: level_name.

Назначение: уровень прилежания персонажа.

Бизнес-смысл: влияет на задержки в выполнении задач (avg_task_delay_days).

Основные сущности предметной области

Таблица Rule

Атрибуты: rule_id, title, description, frequency_id, priority_id, trigger_condition.

Первичный ключ: rule_id.

Семантический кандидатный ключ: title (при условии уникальности).

Назначение: бизнес-правила ухода за планетой.

Бизнес-смысл: «Встал поутру — приведи в порядок планету»: связывает частоту, приоритет, условие.

Таблица Planet

Атрибуты: planet_id, name, size_id, is_infected, integrity_status_id.

Первичный ключ: planet_id.

Семантический кандидатный ключ: name.

Назначение: планета как объект управления.

Бизнес-смысл: хранит размер, статус заражения, целостность — основа для оценки рисков.

Таблица Action

Атрибуты: action_id, name, difficulty_id, urgency_id, consequence_if_skipped.

Первичный ключ: action_id.

Семантический кандидатный ключ: name.

Назначение: действия, которые может выполнить персонаж.

Бизнес-смысл: «Выполоть», «Осмотреть», «Полить» — имеют сложность, срочность, последствия.

Таблица Seed

Атрибуты: seed_id, species_name, category_id, state_id, planet_id.

Первичный ключ: seed_id.

Семантический кандидатный ключ: {species_name, planet_id} (семантика: вид семени на конкретной планете).

Назначение: конкретный экземпляр семени на планете.

Бизнес-смысл: связывает вид, категорию, состояние и планету — источник будущих растений.

Таблица Character

Атрибуты: character_id, name, role_id, diligence_level_id, current_planet_id.

Первичный ключ: character_id.

Семантический кандидатный ключ: name.

Назначение: персонаж, ухаживающий за планетой.

Бизнес-смысл: имеет роль, уровень прилежания, текущую планету — субъект действий.

Таблица Plant

Атрибуты: plant_id, name, category_id, growth_stage_id, is_recognizable, root_penetration_level, seed_id, planet_id, max_safe_harmful_plants.

Первичный ключ: plant_id.

Семантический кандидатный ключ: {name, planet_id}.

Назначение: растущее растение на планете.

Бизнес-смысл: ключевая сущность: баобаб или роза; стадия роста, узнаваемость, уровень проникновения корней.

Журналы и аналитика

Таблица TaskLog

Атрибуты: log_id, character_id, action_id, plant_id, planet_id, executed_at.

Первичный ключ: log_id.

Семантический кандидатный ключ: {character_id, action_id, plant_id, executed_at} (журнал событий).

Назначение: журнал выполненных действий.

Бизнес-смысл: аудит: кто, что, когда сделал; нужен для анализа прилежания и эффективности.

Таблица RiskCase

Атрибуты: case_id, planet_id, description, outcome_id, lessons_learned.

Первичный ключ: case_id.

Семантический кандидатный ключ: {planet_id, description, outcome_id} (уникальный кейс).

Назначение: зарегистрированные инциденты.

Бизнес-смысл: документирование случаев: что произошло, какой исход, какие уроки извлечены.

Таблица SpeciesSimilarity

Атрибуты: similarity_id, species_1, species_2, confusion_risk_id, distinguishable_at_stage.

Первичный ключ: similarity_id.

Семантический кандидатный ключ: {species_1, species_2}.

Назначение: справочник похожих видов.

Бизнес-смысл: помогает персонажу не перепутать баобаб с розой на ранней стадии.

Таблица ThreatLevel

Атрибуты: threat_id, growth_stage_id, planet_size_id, risk_level_id, recommended_action.

Первичный ключ: threat_id.

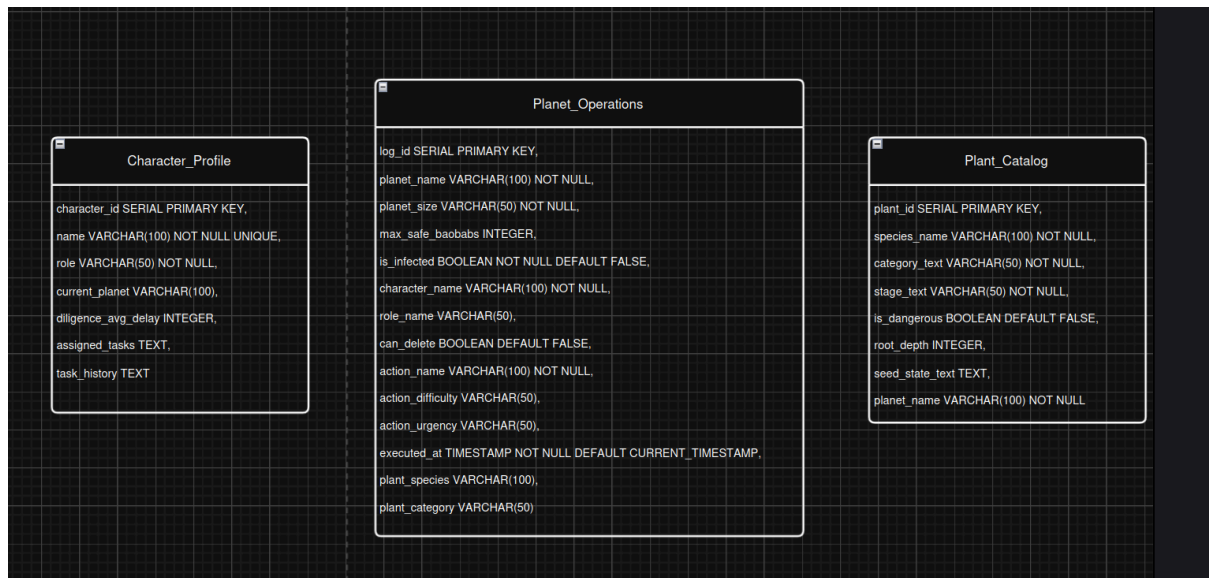
Семантический кандидатный ключ: {growth_stage_id, planet_size_id} (комбинация стадии и размера определяет угрозу).

Назначение: матрица угроз.

Бизнес-смысл: сочетание стадии роста и размера планеты определяет рекомендуемое действие.

2. Неудачное проектное решение

В качестве заведомо неудачного архитектурного решения рассматривается начальная версия схемы, сформированная без предварительного анализа жизненного цикла сущностей, когда казалось, что будет неплохой идеей хранить не ключи, а имена других сущностей для читаемости. Ниже приведен состав неудачных отношений, выявленные ошибки проектирования и конкретные примеры возникающих аномалий.



2.1. Описание неудачных отношений

Таблица Planet_Operations

Атрибуты: log_id, planet_name, planet_size, max_safe_baobabs, is_infected, character_name, role_name, can_delete, action_name, action_difficulty, action_urgency, executed_at, plant_species, plant_category.

Первичный ключ: log_id.

Кандидатный ключ: {planet_name, executed_at}.

Назначение: единый журнал действий по уходу за планетой с встроенными справочными данными.

Бизнес-смысл: фиксация того, кто, когда и что сделал на конкретной планете, с дублированием параметров для быстрого чтения отчетов.

Таблица Plant_Catalog

Атрибуты: plant_id, species_name, category_text, stage_text, is_dangerous, root_depth, seed_state_text, planet_name.

Первичный ключ: plant_id.

Кандидатный ключ: {species_name, planet_name}.

Назначение: реестр всех растений на планетах.

Бизнес-смысл: хранение характеристик растений для контроля их роста и опасности, но с прямым указанием текстовых категорий и состояний без вынесения в справочники.

Таблица Character_Profile

Атрибуты: character_id, name, role, current_planet, diligence_avg_delay, assigned_tasks, task_history.

Первичный ключ: character_id.

Кандидатный ключ: name.

Назначение: профиль персонажа с привязкой к текущей планете и списку задач.

Бизнес-смысл: хранение данных о прилежании персонажа и перечне его обязанностей в виде строки для упрощения интерфейса.

2.2. Ошибки проектирования

Таблица Planet_Operations

1. Нарушение третьей нормальной формы. Атрибуты, описывающие планету (planet_size, max_safe_baobabs, is_infected), зависят от planet_name, а не от первичного ключа log_id. Аналогично, параметры действия (action_difficulty, action_urgency) зависят от action_name. Это создает транзитивные зависимости и избыточное дублирование данных в каждой записи журнала.
2. Смешение сущностей с независимым жизненным циклом. Справочные данные о действиях и параметрах планеты изменяются редко, тогда как журнал растет ежедневно. Их объединение в одном отношении блокирует независимое управление справочниками и усложняет аудит.
3. Отсутствие ссылочной целостности. Поля character_name и action_name хранятся как текст, что позволяет вводить произвольные значения без контроля уникальности и соответствия справочникам ролей или действий.

Таблица Plant_Catalog

1. Нарушение первой нормальной формы. Атрибут seed_state_text содержит неатомарные данные (например, спящее, требует проверки, планета А). Это делает невозможным корректный поиск, фильтрацию и агрегацию по состояниям семян.
2. Транзитивная зависимость категории и стадии. Атрибуты category_text и stage_text зависят не от plant_id напрямую, а от семантического смысла вида растения. При изменении классификации вида потребуется массовое обновление всех записей с ручным поиском по текстовым полям.
3. Логическая несогласованность. Поле planet_name дублируется в каждой строке. При переименовании планеты или ее уничтожении данные рассинхронизируются, а внешние связи с другими таблицами отсутствуют.

Таблица Character_Profile

1. Нарушение первой и второй нормальных форм. Атрибуты assigned_tasks и task_history хранят списки задач в текстовом формате. Это нарушает требование атомарности и создает повторяющиеся группы, что противоречит принципам реляционной модели.
2. Частичная зависимость атрибутов прилежания. Поле diligence_avg_delay зависит от уровня прилежания персонажа, а не напрямую от character_id. В текущей структуре оно встроено в профиль, что при изменении глобальных нормативов прилежания потребует пересчета и обновления множества записей.
3. Циклическая логическая ссылка. Текущая планета персонажа (current_planet) указывается текстом, а журнал действий ссылается на персонажа. При перемещении персонажа на другую планету без каскадного обновления возникает рассинхронизация местоположения.

2.3. Демонстрация аномалий на примерах SQL-операций

Аномалия 1.

Невозможность вставки справочных данных.

Задача: зарегистрировать новое действие “Осмотр корневой системы” в системе до того, как оно будет выполнено кем-либо на конкретной планете.

SQL-операция:

```
INSERT INTO Planet_Operations (action_name, action_difficulty, action_urgency) VALUES ('Осмотр корневой системы', 'Средняя', 'Высокая');
```

```
planet_db=#
INSERT INTO Planet_Operations (action_name, action_difficulty, action_urgency) VALUES ('Осмотр корневой системы', 'Средняя', 'Высокая');
ERROR: null value in column "planet_name" of relation "planet_operations" violates not-null constraint
DETAIL: Failing row contains (1, null, null, null, f, null, null, f, Осмотр корневой системы, Средняя, Высокая, 2026-05-04 09:40:34.737294, null, null).
```

Результат и проблема: Операция завершается ошибкой нарушения ограничения NOT NULL для столбца planet_name. Это происходит потому, что в неудачном проекте таблица Planet_Operations объединяет справочные данные о действиях с фактическими записями о выполненных операциях. Все атрибуты, включая planet_name, character_name, executed_at, объявлены как NOT NULL, так как предполагается, что каждая запись описывает конкретное свершившееся событие.

Аномалия 2.

Рассинхронизация при изменении статуса планеты

Задача: изменить признак заражения планеты “Астероид В-612” на TRUE, так как в почве обнаружены семена баобабов. В журнале хранится 120 записей о прошлых действиях.

SQL-операция:

UPDATE Planet_Operations SET is_infected = TRUE WHERE planet_name = 'Астероид В-612';

Результат и проблема: Запрос обновит все строки, включая архивные. В отчетах появится ложная информация о том, что планета была заражена еще до момента обнаружения семян. Если обновление выполнено с ошибкой или частичным условием, часть строк сохранит FALSE. Возникает логическое противоречие: в одной таблице одна планета одновременно заражена и не заражена. Автоматический контроль угроз перестает работать достоверно.

Аномалия 3.

Потеря данных при очистке журнала (Аномалия удаления)

Задача: удалить архивные записи журнала за прошлый год для снижения объема хранилища.

SQL-операция:

DELETE FROM Planet_Operations WHERE executed_at < '2025-01-01' AND planet_name = 'Астероид В-612';

Результат и проблема: Вместе с архивными записями безвозвратно исчезнут эталонные характеристики планеты (размер, допустимое количество баобабов, статус), так как они хранились только в строках журнала. Сами планеты и персонажи никуда не делись, но система теряет базовые метрики для текущих расчетов рисков. Восстановление потребует ручного ввода или обращения к внешним источникам.

Аномалия 4.

Невозможность корректного обновления состояния семян (Нарушение 1НФ)

Задача: изменить состояние семени с спящее на прорастает в таблице Plant_Catalog.

SQL-операция:

UPDATE Plant_Catalog SET seed_state_text = REPLACE(seed_state_text, 'спящее', 'прорастает') WHERE species_name = 'Баобаб';

Результат и проблема: Операция заменит подстроку во всех записях, содержащих слово спящее, даже если речь идет о разных семенах с составным описанием. При добавлении нового семени без строгого формата строки возникнут несогласованные записи. Поиск и агрегация по состояниям становятся невозможными без сложного парсинга текста на уровне приложения.

3. Определение функциональных зависимостей

Таблица Difficulty

ФЗ: $\text{difficulty_id} \rightarrow \{\text{difficulty_name}, \text{success_rate_modifier}\}$; $\text{difficulty_name} \rightarrow \{\text{difficulty_id}, \text{success_rate_modifier}\}$

Кандидатные ключи: difficulty_id , difficulty_name

Обоснование и содержательный анализ: Уровень сложности действия однозначно определяется либо суррогатным идентификатором, либо его названием. В предметной области каждое название сложности (например, «Требуется усилий», «Крайне опасно») соответствует строго определенному модификатору вероятности успеха. Обратная зависимость также выполняется, так как названия не дублируются.

Минимальное покрытие: $\{\text{difficulty_id} \rightarrow \text{difficulty_name}, \text{difficulty_id} \rightarrow \text{success_rate_modifier}, \text{difficulty_name} \rightarrow \text{difficulty_id}, \text{difficulty_name} \rightarrow \text{success_rate_modifier}\}$

Нарушения нормальных форм: Отсутствуют. Все детерминанты являются кандидатными ключами.

Результат: Отношение находится в BCNF.

Таблица Urgency

ФЗ: $\text{urgency_id} \rightarrow \{\text{urgency_name}, \text{max_delay_hours}\}$; $\text{urgency_name} \rightarrow \{\text{urgency_id}, \text{max_delay_hours}\}$

Кандидатные ключи: urgency_id , urgency_name

Обоснование: Срочность действия определяется его названием и жестко привязана к допустимому времени задержки. Например, категория «Неотложно» семантически подразумевает нулевое или минимальное значение max_delay_hours . Зависимость прямая, транзитивных связей нет.

Минимальное покрытие: $\{\text{urgency_id} \rightarrow \text{urgency_name}, \text{urgency_id} \rightarrow \text{max_delay_hours}, \text{urgency_name} \rightarrow \text{urgency_id}, \text{urgency_name} \rightarrow \text{max_delay_hours}\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Category

ФЗ: $\text{category_id} \rightarrow \{\text{category_name}, \text{is_dangerous}\}$; $\text{category_name} \rightarrow \{\text{category_id}, \text{is_dangerous}\}$

Кандидатные ключи: category_id , category_name

Обоснование: Категория растения определяет его принадлежность к полезным или вредным видам. В мире Маленького принца это фундаментальное разделение: если категория «Баобаб» или «Сорная трава», флаг is_dangerous всегда принимает значение

TRUE. Название категории однозначно определяет этот флаг, а суррогатный ключ — все остальные атрибуты.

Минимальное покрытие: {category_id → category_name, category_id → is_dangerous, category_name → category_id, category_name → is_dangerous}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица GrowthStage

ФЗ: stage_id → {stage_name, stage_order}; stage_name → {stage_id, stage_order}; stage_order → {stage_id, stage_name}

Кандидатные ключи: stage_id, stage_name, stage_order

Обоснование: Стадия роста растения может быть идентифицирована по номеру, названию или порядковому номеру в цикле развития. В предметной области «Спящее семя», «Росток», «Укоренившееся растение» имеют четкую последовательность.

Каждый из этих атрибутов функционально определяет остальные, так как в рамках одной классификации дублирование порядка или названий не допускается.

Минимальное покрытие: {stage_id → stage_name, stage_id → stage_order, stage_name → stage_id, stage_name → stage_order, stage_order → stage_id, stage_order → stage_name}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Planet

ФЗ: planet_id → {name, size_id, is_infected, integrity_status_id}; name → {planet_id, size_id, is_infected, integrity_status_id}

Кандидатные ключи: planet_id, name

Обоснование: Планета идентифицируется по уникальному ID или названию (например, «Астероид Б-612»). Ее физический размер, статус заражения почвы и общая целостность являются характеристиками, которые изменяются независимо, но в каждый момент времени однозначно определяются самой планетой. Внешние ключи ссылаются на справочники PlanetSize и IntegrityStatus, но не создают транзитивных зависимостей внутри данного отношения, так как хранят только идентификаторы, а не описательные атрибуты.

Минимальное покрытие: {planet_id → name, planet_id → size_id, planet_id → is_infected, planet_id → integrity_status_id, name → planet_id, name → size_id, name → is_infected, name → integrity_status_id}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Seed

ФЗ: seed_id → {species_name, category_id, state_id, planet_id}; {species_name, planet_id} → {seed_id, category_id, state_id}

Кандидатные ключи: seed_id, {species_name, planet_id}

Обоснование: Конкретный экземпляр семени уникален по ID. Семантически семя также однозначно определяется комбинацией вида растения и планеты, на которой оно находится в почве. Категория и текущее состояние семени (спящее, готово к прорастанию) зависят от конкретного экземпляра. В предметной области одно и то же семя не может одновременно находиться на разных планетах, поэтому составной ключ из вида и планеты валиден.

Минимальное покрытие: {seed_id → species_name, seed_id → category_id, seed_id → state_id, seed_id → planet_id, species_name → seed_id, planet_id → seed_id, species_name → category_id, species_name → state_id, planet_id → category_id, planet_id → state_id}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Plant

ФЗ: plant_id → {name, category_id, growth_stage_id, is_recognizable, root_penetration_level, seed_id, planet_id, max_safe_harmful_plants}; {name, planet_id} → {plant_id, category_id, growth_stage_id, is_recognizable, root_penetration_level, seed_id, max_safe_harmful_plants}

Кандидатные ключи: plant_id, {name, planet_id}

Обоснование: Растение уникально идентифицируется по ID или по комбинации названия и планеты (поскольку на разных планетах могут расти растения с одинаковыми именами). Стадия роста, уровень проникновения корней, возможность распознавания и связь с исходным семенем являются физическими характеристиками конкретного экземпляра. Атрибут max_safe_harmful_plants в данной схеме хранится как характеристика растения, отражающая его индивидуальную опасность для почвы, и функционально зависит от plant_id.

Минимальное покрытие: Все атрибуты неключевого типа зависят от первичного ключа.

Составные зависимости разбиваются на атомарные правые части.

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Character

ФЗ: character_id → {name, role_id, diligence_level_id, current_planet_id}; name → {character_id, role_id, diligence_level_id, current_planet_id}

Кандидатные ключи: character_id, name

Обоснование: Персонаж уникален по ID или имени. Его роль в сообществе, врожденный или приобретенный уровень прилежания, а также текущее

местоположение являются атрибутами, описывающими состояние персонажа в конкретный момент времени. В предметной области «Встал поутру — приведи в порядок свою планету» подразумевает, что `current_planet_id` может меняться, но в рамках одной записи жестко определен самим персонажем. Зависимость прямая, транзитивных связей внутри отношения нет.

Минимальное покрытие: $\{character_id \rightarrow name, character_id \rightarrow role_id, character_id \rightarrow diligence_level_id, character_id \rightarrow current_planet_id, name \rightarrow character_id, name \rightarrow role_id, name \rightarrow diligence_level_id, name \rightarrow current_planet_id\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица TaskLog

ФЗ: $log_id \rightarrow \{character_id, action_id, plant_id, planet_id, executed_at\}; \{character_id, action_id, plant_id, executed_at\} \rightarrow \{log_id, planet_id\}$

Кандидатные ключи: $log_id, \{character_id, action_id, plant_id, executed_at\}$

Обоснование и содержательный анализ: Журнал фиксирует факт выполнения задачи.

Каждая запись уникальна по суррогатному ключу. Семантически событие однозначно определяется тем, кто выполнил действие, над каким растением и в какой момент времени. Однако здесь выявляется важная структурная особенность: атрибут `planet_id` функционально зависит от `plant_id` (поскольку каждое растение растет строго на одной планете: $plant_id \rightarrow planet_id$). Следовательно, в журнале возникает транзитивная зависимость: $log_id \rightarrow plant_id \rightarrow planet_id$. Атрибут `planet_id` дублирует информацию, которая уже содержится в связи с растением.

Минимальное покрытие: $\{log_id \rightarrow character_id, log_id \rightarrow action_id, log_id \rightarrow plant_id, log_id \rightarrow executed_at, plant_id \rightarrow planet_id\}$

Нарушения нормальных форм: Выявлена транзитивная зависимость $plant_id \rightarrow planet_id$, где `plant_id` не является суперключом таблицы TaskLog.

Результат: Отношение не находится в BCNF. Отношение находится в 1NF.

Таблица SpeciesSimilarity

ФЗ: $similarity_id \rightarrow \{species_1, species_2, confusion_risk_id, distinguishable_at_stage\}; \{species_1, species_2\} \rightarrow \{similarity_id, confusion_risk_id, distinguishable_at_stage\}$

Кандидатные ключи: $similarity_id, \{species_1, species_2\}$

Обоснование: Риск путаницы определяется парой видов. В предметной области молодые ростки баобаба и розы практически идентичны, и только на определенной стадии их можно различить. Пара видов однозначно определяет уровень риска и стадию различимости. Порядок записи видов в схеме фиксирован, поэтому составной ключ валиден.

Минимальное покрытие: $\{similarity_id \rightarrow species_1, similarity_id \rightarrow species_2, similarity_id \rightarrow confusion_risk_id, similarity_id \rightarrow distinguishable_at_stage, species_1 \rightarrow similarity_id, species_2 \rightarrow similarity_id, species_1 \rightarrow confusion_risk_id, species_2 \rightarrow confusion_risk_id\}$

confusion_risk_id, species_1 → distinguishable_at_stage, species_2 → distinguishable_at_stage}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица ThreatLevel

ФЗ: threat_id → {growth_stage_id, planet_size_id, risk_level_id, recommended_action};
{growth_stage_id, planet_size_id} → {threat_id, risk_level_id, recommended_action}

Кандидатные ключи: threat_id, {growth_stage_id, planet_size_id}

Обоснование: Уровень угрозы формируется на основе сочетания стадии роста растения и размера планеты. В мире Маленького принца правило гласит: «Если планета очень маленькая, а баобабов много, они разорвут ее на клочки». Следовательно, комбинация стадии и размера однозначно определяет рекомендуемое действие и классификацию риска. Зависимость прямая, без транзитивных переходов.

Минимальное покрытие: {threat_id → growth_stage_id, threat_id → planet_size_id, threat_id → risk_level_id, threat_id → recommended_action, growth_stage_id → threat_id, planet_size_id → threat_id, growth_stage_id → risk_level_id, planet_size_id → risk_level_id, growth_stage_id → recommended_action, planet_size_id → recommended_action}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Priority

ФЗ: priority_id → {priority_name}

Обоснование: priority_id — первичный ключ. Название приоритета функционально определяется его идентификатором. В предметной области каждый уровень приоритета (например, «Высокий», «Средний», «Низкий») уникален и не дублируется. Обратная зависимость priority_name → priority_id также выполняется, так как названия приоритетов не повторяются.

Минимальное покрытие: {priority_id → priority_name, priority_name → priority_id}

Нарушения нормальных форм: Отсутствуют. Оба детерминанта являются кандидатными ключами.

Результат: Отношение находится в BCNF.

Таблица Frequency

ФЗ: frequency_id → {frequency_name, interval_days}; frequency_name → {frequency_id, interval_days}

Обоснование: frequency_id — первичный ключ. Частота выполнения правил определяется либо суррогатным идентификатором, либо названием (например,

«Ежедневно», «Еженедельно»). Интервал в днях функционально зависит от частоты: если частота «Ежедневно», то `interval_days = 1`; если «Еженедельно», то `interval_days = 7`. В предметной области каждое название частоты уникально и соответствует строго определенному интервалу.

Минимальное покрытие: $\{frequency_id \rightarrow frequency_name, frequency_id \rightarrow interval_days, frequency_name \rightarrow frequency_id, frequency_name \rightarrow interval_days\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Role

ФЗ: $role_id \rightarrow \{role_name, is_available, can_delete_harmful_plants\}; role_name \rightarrow \{role_id, is_available, can_delete_harmful_plants\}$

Обоснование: `role_id` — первичный ключ. Роль персонажа (например, «Садовник», «Наблюдатель», «Старший смотритель») определяет его доступность и право удалять вредные растения. В предметной области «Есть такое твердое правило» — только определенные роли могут выполнять критические действия по удалению баобабов.

Название роли уникально и функционально определяет все остальные атрибуты.

Минимальное покрытие: $\{role_id \rightarrow role_name, role_id \rightarrow is_available, role_id \rightarrow can_delete_harmful_plants, role_name \rightarrow role_id, role_name \rightarrow is_available, role_name \rightarrow can_delete_harmful_plants\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица SeedState

ФЗ: $state_id \rightarrow \{state_name, is_removable\}; state_name \rightarrow \{state_id, is_removable\}$

Обоснование: `state_id` — первичный ключ. Состояние семени (например, «Спящее», «Прорастает», «Активное») определяет, можно ли его удалить до прорастания. В предметной области семена невидимы и спят глубоко под землей, пока одно из них не вздумает проснуться. Каждое состояние функционально определяет возможность удаления и уникально в рамках классификации.

Минимальное покрытие: $\{state_id \rightarrow state_name, state_id \rightarrow is_removable, state_name \rightarrow state_id, state_name \rightarrow is_removable\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица PlanetSize

ФЗ: $size_id \rightarrow \{size_name, description, max_safe_plants\}; size_name \rightarrow \{size_id, description, max_safe_plants\}$

Обоснование: `size_id` — первичный ключ. Размер планеты (например, «Маленькая», «Средняя», «Большая») функционально определяет описание и максимально

допустимое количество безопасных растений. В предметной области «если планета очень маленькая, а баобабов много, они разорвут ее на клочки» — это означает, что `max_safe_plants` жестко зависит от размера. Название размера уникально и определяет все остальные атрибуты.

Минимальное покрытие: $\{size_id \rightarrow size_name, size_id \rightarrow description, size_id \rightarrow max_safe_plants, size_name \rightarrow size_id, size_name \rightarrow description, size_name \rightarrow max_safe_plants\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица **IntegrityStatus**

ФЗ: $status_id \rightarrow \{status_name, requires_immediate_action\}; status_name \rightarrow \{status_id, requires_immediate_action\}$

Обоснование: `status_id` — первичный ключ. Статус целостности планеты (например, «Нормальный», «Требуется внимания», «Критический») определяет, требуется ли немедленное действие. В предметной области «Непрерывно надо каждый день выпалывать баобабы» — это активируется при определенных статусах. Каждое название статуса уникально и функционально определяет необходимость срочных мер.

Минимальное покрытие: $\{status_id \rightarrow status_name, status_id \rightarrow requires_immediate_action, status_name \rightarrow status_id, status_name \rightarrow requires_immediate_action\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица **Outcome**

ФЗ: $outcome_id \rightarrow \{outcome_name, description\}$

Обоснование: `outcome_id` — первичный ключ. Исход рискованного события (например, «Планета уничтожена», «Баобаб удален вовремя», «Роза сохранена») функционально определяется идентификатором. Описание исхода зависит от названия, но название не обязательно является уникальным кандидатным ключом, так как могут существовать схожие исходы с разными описаниями. Поэтому только `primary key` выступает детерминантом.

Минимальное покрытие: $\{outcome_id \rightarrow outcome_name, outcome_id \rightarrow description\}$

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица **ConfusionRisk**

ФЗ: $risk_id \rightarrow \{risk_name\}$

Обоснование: `risk_id` — первичный ключ. Уровень риска путаницы (например, «Высокий», «Средний», «Низкий») функционально определяется идентификатором. В

предметной области «молодые ростки у них почти одинаковые» — это описывает высокий риск спутать баобаб с розой. Название риска уникально и может выступать кандидатным ключом.

Минимальное покрытие: {risk_id → risk_name, risk_name → risk_id}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица RiskLevel

ФЗ: risk_level_id → {risk_level_name}

Обоснование: risk_level_id — первичный ключ. Уровень риска (например, «Низкий», «Критический») функционально определяется идентификатором. В предметной области это используется для визуализации и приоритезации угроз. Название уровня риска уникально и может выступать кандидатным ключом.

Минимальное покрытие: {risk_level_id → risk_level_name, risk_level_name → risk_level_id}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица DiligenceLevel

ФЗ: level_id → {level_name, avg_task_delay_days}; level_name → {level_id, avg_task_delay_days}

Обоснование: level_id — первичный ключ. Уровень прилежания персонажа (например, «Ответственный», «Лентяй») функционально определяет среднюю задержку в выполнении задач. В предметной области «Я знал одну планету, на ней жил лентяй. Он не выполнил вовремя три кустика» — это демонстрирует, что уровень прилежания напрямую влияет на avg_task_delay_days. Название уровня уникально и определяет все атрибуты.

Минимальное покрытие: {level_id → level_name, level_id → avg_task_delay_days, level_name → level_id, level_name → avg_task_delay_days}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Rule

ФЗ: rule_id → {title, description, frequency_id, priority_id, trigger_condition}; title → {rule_id, description, frequency_id, priority_id, trigger_condition}

Обоснование: rule_id — первичный ключ. Бизнес-правило ухода за планетой уникально идентифицируется либо по ID, либо по названию. В предметной области «Есть такое твердое правило, — сказал мне позднее Маленький принц. — Встал поутру, умылся, привел себя в порядок — и сразу же приведи в порядок свою планету». Название

правила уникально в рамках системы. Частота и приоритет правила функционально зависят от самого правила, так как каждое правило имеет свою периодичность выполнения и степень важности.

Минимальное покрытие: {rule_id → title, rule_id → description, rule_id → frequency_id, rule_id → priority_id, rule_id → trigger_condition, title → rule_id, title → description, title → frequency_id, title → priority_id, title → trigger_condition}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица Action

ФЗ: action_id → {name, difficulty_id, urgency_id, consequence_if_skipped}; name → {action_id, difficulty_id, urgency_id, consequence_if_skipped}

Обоснование: action_id — первичный ключ. Действие, которое может выполнить персонаж (например, «Выполоть баобаб», «Осмотреть почву», «Полить розу»), уникально определяется либо по ID, либо по названию. Сложность и срочность действия функционально зависят от самого действия: «Вырвать баобаб» сложнее и срочнее, чем «полить розу». Последствия пропуска также определяются конкретным действием.

Минимальное покрытие: {action_id → name, action_id → difficulty_id, action_id → urgency_id, action_id → consequence_if_skipped, name → action_id, name → difficulty_id, name → urgency_id, name → consequence_if_skipped}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

Таблица RiskCase

ФЗ: case_id → {planet_id, description, outcome_id, lessons_learned}

Обоснование: case_id — первичный ключ. Зарегистрированный инцидент уникально идентифицируется по ID. Описание случая, ссылка на планету, исход и извлеченные уроки функционально зависят от конкретного кейса. В предметной области каждый случай уникален: «Я знал одну планету, на ней жил лентяй» — это конкретный кейс со своим описанием и последствиями. Составные кандидатные ключи отсутствуют, так как описание может повторяться, а одна планета может иметь несколько инцидентов.

Минимальное покрытие: {case_id → planet_id, case_id → description, case_id → outcome_id, case_id → lessons_learned}

Нарушения нормальных форм: Отсутствуют.

Результат: Отношение находится в BCNF.

4. Приведение отношений к третьей нормальной форме (3NF)

Третья нормальная форма требует полного отсутствия частичных и транзитивных зависимостей неключевых атрибутов от первичного ключа. На основе анализа функциональных зависимостей, выполненного в пункте 3, большинство отношений исходной схемы уже удовлетворяют требованиям 3NF благодаря использованию суррогатных первичных ключей и четкому разделению справочных, бизнес- и журнальных данных.

4.1. Доказательство соответствия 3NF для справочных отношений

В данную группу входят таблицы: Difficulty, Urgency, Priority, Frequency, Category, Role, GrowthStage, SeedState, PlanetSize, IntegrityStatus, Outcome, ConfusionRisk, RiskLevel, DiligenceLevel.

Доказательство для всех отношений группы строится по единому алгоритму:

1. Первая нормальная форма (1НФ): Все атрибуты в таблицах являются атомарными. Значения не содержат списков, массивов или структурированных объектов. Каждая запись описывает один уникальный параметр справочника без повторяющихся групп.
2. Вторая нормальная форма (2НФ): Первичные ключи во всех справочных таблицах являются простыми (суррогатные идентификаторы типа SERIAL). По определению, частичная зависимость неключевого атрибута от части составного ключа невозможна, так как ключ состоит из одного атрибута.
3. Третья нормальная форма (3НФ): Все неключевые атрибуты (например, difficulty_name, max_delay_hours, is_dangerous, stage_order, avg_task_delay_days) зависят исключительно от первичного ключа. Транзитивные зависимости отсутствуют, так как ни один неключевой атрибут не определяет другой неключевой атрибут в рамках одного отношения. Например, в таблице GrowthStage атрибут stage_order определяет порядок стадий, но не определяет stage_name напрямую без участия ключа, что исключает цепочки вида ключ → неключ → неключ.

Вывод: Все справочные отношения находятся в 3NF. Декомпозиция не требуется. Структура таблиц сохраняется в исходном виде.

4.2. Доказательство соответствия 3NF для основных бизнес-отношений

В данную группу входят таблицы: Rule, Planet, Action, Seed, Character, Plant.

Доказательство:

1. 1НФ: Атрибуты атомарны. Внешние ключи хранят только скалярные идентификаторы связанных сущностей, не нарушая требование неделимости данных.
2. 2НФ: Простые первичные ключи исключают частичные зависимости.
3. 3НФ: Проверка на транзитивные зависимости. В данных таблицах присутствуют внешние ключи (например, `size_id` и `integrity_status_id` в Planet, `category_id` и `growth_stage_id` в Plant). Эти атрибуты не являются детерминантами для других неключевых полей внутри своих таблиц. Они лишь ссылаются на соответствующие справочники. Все описательные атрибуты (названия, флаги состояния, числовые параметры вроде `root_penetration_level`) зависят напрямую от первичного ключа отношения. Цепочек вида первичный_ключ → внешний_ключ → описательный_атрибут внутри одного отношения не обнаружено.

Вывод: Основные бизнес-отношения соответствуют 3NF. Структура сохраняется без изменений.

4.3. Доказательство соответствия 3NF для аналитических отношений

В данную группу входят таблицы: RiskCase, SpeciesSimilarity, ThreatLevel.

Доказательство:

1. 1НФ: Все поля атомарны. В таблице SpeciesSimilarity пара видов хранится в отдельных атрибутах `species_1` и `species_2`, что соответствует требованиям атомарности.
2. 2НФ: Первичные ключи простые (суррогатные id). Частичные зависимости отсутствуют.
3. 3НФ: В таблице RiskCase атрибуты `planet_id` и `outcome_id` являются внешними ключами и не определяют друг друга или текстовые поля `description` и `lessons_learned`. В таблице ThreatLevel комбинация `growth_stage_id` и `planet_size_id` определяет риск, но сами по себе они не являются детерминантами для `recommended_action` без участия второго компонента. Транзитивных зависимостей не выявлено.

Вывод: Аналитические отношения находятся в 3NF. Декомпозиция не требуется.

4.4. Преобразование отношения TaskLog (устранение нарушения 3NF)

Выявленное нарушение: Транзитивная зависимость неключевого атрибута.

Функциональная зависимость, приводящая к декомпозиции: $\text{plant_id} \rightarrow \text{planet_id}$

Обоснование: В исходной схеме таблица TaskLog содержала атрибуты log_id, character_id, action_id, plant_id, planet_id, executed_at. Согласно предметной области, каждое растение растет строго на одной планете, что задает функциональную зависимость $\text{plant_id} \rightarrow \text{planet_id}$. При этом журнал задач связывается с растением через plant_id, то есть выполняется цепочка $\text{log_id} \rightarrow \text{plant_id} \rightarrow \text{planet_id}$. Поскольку plant_id не является суперключом таблицы TaskLog, возникает транзитивная зависимость, нарушающая условие 3NF. Хранение planet_id напрямую в журнале создает избыточность и риски рассинхронизации.

Процесс декомпозиции:

Для устранения транзитивной зависимости отношение TaskLog разбивается путем выноса зависимого атрибута planet_id. Атрибут planet_id исключается из журнала, так как связь с планетой уже опосредована через plant_id и таблицу Plant.

Полученные отношения после преобразования:

1. TaskLog(log_id, character_id, action_id, plant_id, executed_at)

Первичный ключ: log_id

Состав: фиксирует факт выполнения задачи, исполнителя, тип действия, объект воздействия и время.

2. Plant(plant_id, name, category_id, growth_stage_id, is_recognizable, root_penetration_level, seed_id, planet_id, max_safe_harmful_plants)

Первичный ключ: plant_id

Состав: остается без изменений, так как именно здесь должна храниться привязка растения к планете.

Устраненные аномалии в результате преобразования:

- Аномалия обновления: В исходной схеме при изменении планеты, на которой находится растение, требовалось обновлять все связанные записи в журнале действий.

Теперь информация о привязке хранится в одном месте (таблица Plant), что исключает риск частичного обновления и логических противоречий.

- Аномалия вставки: Ранее невозможно было корректно добавить запись в журнал без явного дублирования идентификатора планеты. Теперь журнал фиксирует только факт выполнения задачи над конкретным растением, а планета определяется автоматически через соединение с таблицей Plant.

- Избыточность хранения: Атрибут planet_id больше не дублируется в каждой строке журнала. Это снижает объем занимаемого дискового пространства, ускоряет операции модификации и упрощает поддержку ссылочной целостности.

5. Изменения в функциональных зависимостях после приведения к 3NF

Проведенный в предыдущих разделах анализ показал, что большинство отношений уже соответствовали требованиям третьей нормальной формы, так как использовали суррогатные первичные ключи, а справочные и бизнес-данные были разделены логически. Единственным отношением, потребовавшим структурного преобразования, стала таблица журнала выполненных задач. Ниже подробно описаны изменения в структуре функциональных зависимостей, произошедшие в результате нормализации.

5.1. Локализация функциональных зависимостей в отдельных отношениях

В ходе декомпозиции таблицы журнала задач была выявлена и перенесена в соответствующее отношение зависимость, описывающая принадлежность растения к планете. Изначально в журнале присутствовала функциональная зависимость, согласно которой идентификатор растения функционально определял идентификатор планеты. Эта зависимость была локализована в таблице растений, где идентификатор растения выступает первичным ключом. В результате атрибут, описывающий планету, перестал храниться в журнале задач и остался только в таблице растений. Таким образом, зависимость, отражающая бизнес-правило «каждое растение растет строго на одной планете», теперь целиком сосредоточена в Plant, а не в TaskLog

5.2. Зависимости, переставшие вызывать нарушения нормальных форм

До преобразования в журнале задач существовала транзитивная цепочка зависимостей. Первичный ключ журнала определял идентификатор растения, а идентификатор растения, в свою очередь, определял идентификатор планеты. Поскольку идентификатор растения не являлся суперключом таблицы журнала, возникала транзитивная зависимость неключевого атрибута от первичного ключа через другой неключевой атрибут. Это нарушало условие третьей нормальной формы. После удаления избыточного атрибута планеты из журнала транзитивная цепочка была разорвана. Все оставшиеся атрибуты журнала теперь зависят непосредственно и полностью от первичного ключа записи. Нарушение, вызванное косвенной зависимостью, полностью устранено, и отношение приведено в соответствие с требованиями 3NF.

5.3. Изменение набора зависимостей в полученной схеме

Глобальный набор функциональных зависимостей предметной области не изменился.

5.4. Сохранение зависимостей после декомпозиции

Выполненная декомпозиция таблицы журнала задач обладает свойством сохранения зависимостей. Это означает, что ни одно из исходных бизнес-правил не было утеряно в процессе нормализации. Зависимость, определяющая привязку растения к планете, по-прежнему действует в системе, но теперь она проверяется и поддерживается на уровне таблицы растений, где это семантически корректно. Кроме того, декомпозиция выполнена без потерь соединения. Исходные данные могут быть полностью восстановлены путем естественного соединения таблицы журнала задач с таблицей растений по общему атрибуту идентификатора растения. При соединении не возникает ни лишних строк, ни потери информации. Все ограничения целостности, существовавшие в исходной схеме, перенесены в нормализованную структуру без изменений, что гарантирует полную эквивалентность новой схемы исходной предметной области.

6. Проверка схемы на соответствие BCNF

Нормальная форма Бойса-Кодда (BCNF) является более строгой версией третьей нормальной формы. Отношение находится в BCNF тогда и только тогда, когда в каждой нетривиальной функциональной зависимости $X \rightarrow Y$ детерминант X является потенциальным ключом (суперключом) данного отношения.

На основании анализа функциональных зависимостей, проведенного в пункте 3, и преобразований, выполненных в пункте 4, проведем проверку всех отношений схемы на соответствие BCNF.

Справочные отношения

К данной группе относятся таблицы: Difficulty, Urgency, Priority, Frequency, Category, Role, GrowthStage, SeedState, PlanetSize, IntegrityStatus, Outcome, ConfusionRisk, RiskLevel, DiligenceLevel.

Анализ:

Во всех справочных таблицах первичным ключом выступает суррогатный идентификатор (например, difficulty_id, category_id). Этот идентификатор функционально определяет все остальные атрибуты таблицы. В ряде случаев (например, Category, GrowthStage, PlanetSize) существуют альтернативные кандидатные ключи, состоящие из уникальных текстовых или числовых полей (category_name, stage_order, size_name).

Пример для таблицы Category:

Функциональные зависимости:

$\text{category_id} \rightarrow \{\text{category_name}, \text{is_dangerous}\}$

$\text{category_name} \rightarrow \{\text{category_id}, \text{is_dangerous}\}$

В данном случае детерминантами выступают category_id и category_name. Оба этих атрибута являются кандидатными ключами отношения. Следовательно, условие BCNF выполняется: любой детерминант является суперключом.

Аналогичная логика применима ко всем остальным справочным таблицам схемы. В них отсутствуют неключевые атрибуты, которые определяли бы другие атрибуты, не являясь при этом уникальными идентификаторами записей.

Результат: Все справочные отношения находятся в BCNF.

Основные бизнес-отношения

К данной группе относятся таблицы: Rule, Planet, Action, Seed, Plant, Character.

Анализ:

Таблица Planet

Функциональные зависимости:

$\text{planet_id} \rightarrow \{\text{name}, \text{size_id}, \text{is_infected}, \text{integrity_status_id}\}$

$\text{name} \rightarrow \{\text{planet_id}, \text{size_id}, \text{is_infected}, \text{integrity_status_id}\}$

Детерминантами являются planet_id (первичный ключ) и name (уникальное имя планеты). Оба атрибута являются кандидатными ключами. Транзитивных зависимостей, где детерминант не является ключом, не выявлено.

Результат: Таблица находится в BCNF.

Таблица Plant

Функциональные зависимости:

$\text{plant_id} \rightarrow \{\text{name}, \text{category_id}, \text{growth_stage_id}, \text{is_recognizable}, \text{root_penetration_level}, \text{seed_id}, \text{planet_id}, \text{max_safe_harmful_plants}\}$

$\{\text{name}, \text{planet_id}\} \rightarrow \{\text{plant_id}, \text{category_id}, \text{growth_stage_id}, \text{is_recognizable}, \text{root_penetration_level}, \text{seed_id}, \text{max_safe_harmful_plants}\}$

Детерминантами выступают plant_id (первичный ключ) и составной ключ {name, planet_id} (семантическая уникальность растения на конкретной планете). Все детерминанты являются суперключами.

Результат: Таблица находится в BCNF.

Таблица Character

Функциональные зависимости:

$\text{character_id} \rightarrow \{\text{name}, \text{role_id}, \text{diligence_level_id}, \text{current_planet_id}\}$

$\text{name} \rightarrow \{\text{character_id}, \text{role_id}, \text{diligence_level_id}, \text{current_planet_id}\}$

Детерминантами являются character_id и name. Оба атрибута являются кандидатными ключами.

Результат: Таблица находится в BCNF.

Аналогичная проверка для таблиц Rule, Action и Seed подтверждает, что все их детерминанты являются первичными или альтернативными ключами. Транзитивные зависимости отсутствуют.

Журналы и аналитические отношения

К данной группе относятся таблицы: TaskLog, SpeciesSimilarity, ThreatLevel.

Анализ:

Таблица TaskLog (после нормализации)

В пункте 4 из данного отношения был удален атрибут planet_id для устранения транзитивной зависимости plant_id → planet_id.

Оставшиеся функциональные зависимости:

log_id → {character_id, action_id, plant_id, executed_at}

Единственным детерминантом неключевых атрибутов является log_id (первичный ключ). Атрибут plant_id теперь является только внешним ключом, не определяющим другие поля внутри журнала.

Результат: Таблица приведена в BCNF.

Таблица ThreatLevel

Функциональные зависимости:

threat_id → {growth_stage_id, planet_size_id, risk_level_id, recommended_action}
{growth_stage_id, planet_size_id} → {threat_id, risk_level_id, recommended_action}

Детерминантами выступают threat_id (первичный ключ) и составной ключ {growth_stage_id, planet_size_id}, который определяет уровень угрозы. Оба детерминанта являются суперключами. Атрибут risk_level_id не является детерминантом, так как один и тот же уровень риска может соответствовать разным комбинациям стадий и размеров планет.

Результат: Таблица находится в BCNF.

Таблица SpeciesSimilarity

Функциональные зависимости:

similarity_id → {species_1, species_2, confusion_risk_id, distinguishable_at_stage}
{species_1, species_2} → {similarity_id, confusion_risk_id, distinguishable_at_stage}

Детерминантами являются similarity_id и составной ключ {species_1, species_2}. Оба являются суперключами.

Результат: Таблица находится в BCNF.

7. Сравнение исходной схемы, схем после приведения к 3NF и BCNF, а также предложенные варианты денормализации

7.1. Сопоставление исходной схемы и нормализованных версий

Единственное преобразование, выполненное при приведении к 3NF и BCNF, затронуло таблицу журнала выполненных задач. Из нее был исключен атрибут, содержащий идентификатор планеты, так как он функционально зависел от идентификатора растения, что создавало транзитивную зависимость. После удаления избыточного поля отношение стало полностью соответствовать BCNF. Все остальные таблицы сохранили исходную структуру, так как в них не было обнаружено частичных или транзитивных зависимостей, а все детерминанты являлись суперключами.

Таким образом, схема после нормализации отличается от исходной только устранением одной структурной избыточности.

7.2. Предлагаемые варианты денормализации

Вариант 1.

Включение названия планеты и статуса заражения в журнал действий. Это позволяет формировать отчеты по срочным вмешательствам без выполнения соединений с таблицей планет.

Основным риском является необходимость каскадного обновления всех записей журнала при переименовании планеты или изменении ее статуса, что требует реализации триггерной синхронизации.

Вариант 2.

В таблицу планет вводится целочисленное поле, хранящее текущее количество распознанных баобабов и сорных трав. Поле обновляется автоматически при добавлении или удалении растений. Такой подход устраняет необходимость выполнения агрегирующих запросов с группировкой при проверке бизнес-правила о предельной нагрузке на почву планеты. Недостатком выступает усложнение логики записи: любая операция с растениями должна атомарно корректировать счетчик, иначе возникает риск рассогласования данных.

8. Разработка и реализация триггера на языке PL/pgSQL для системы управления задачами

триггер

менеджер задач

Суть триггера:

При создании новой задачи триггер автоматически проверяет условия и вносит изменения.

- у работника есть ли права на этот проект
 - если нет прав, нужные вставки в логи
- в проекте не перенакопилось неисполненных задач
- проверить бюджет, хватает ли его
- есть ли у пользователя права распределять бюджет
- вставить уведомление
- обновить проект
- добавить логи

Таблицы, которые затрагиваются:

Таблица	Действие
Task	Вставка новой задачи (целевая таблица)
Employee	Чтение лимита задач (для проверки)
Notification	Создание уведомления для исполнителя
Project	Обновление статистики проекта (+1 задача)
BudgetReservation	Резервирование бюджета на задачу
TaskAudit	Запись в лог аудита

8.1. Обоснование выбора триггера и бизнес-правила

Условия, при которых не вставится задача

- у работника нет прав в этом проекте
- проект не активен
- превышен бюджет проекта
- превышен бюджет, которым может управлять работник
- у работника уже достигнут лимит задач в этом проекте

Бизнес-правило, реализуемое триггером: «Новая задача может быть создана только при одновременном соблюдении всех условий:

1. исполнитель имеет права на работу с проектом,
2. в проекте не превышен лимит активных задач у исполнителя,
3. доступен достаточный бюджет на выполнение работы».

Выбор триггера уровня BEFORE INSERT обоснован необходимостью предотвратить вставку некорректной записи в таблицу задач до ее фиксации в базе данных. Это позволяет избежать ситуаций, когда задача создана, но не может быть выполнена из-за нарушения бизнес-ограничений, что потребовало бы отката транзакции или ручного исправления данных.

Уровень обработки FOR EACH ROW выбран потому, что проверка условий должна выполняться индивидуально для каждой создаваемой задачи. Разные задачи могут назначаться разным сотрудникам, на разные проекты, с разными требованиями к бюджету и правам доступа. Пакетная обработка на уровне оператора не позволила бы корректно проверить контекст каждой отдельной записи.

Триггер обрабатывает операцию INSERT, так как именно в момент создания задачи необходимо выполнить все проверки. Операции UPDATE и DELETE не обрабатываются, поскольку изменение или удаление существующей задачи регулируется другими бизнес-правилами и не требует повторной проверки исходных условий назначения.

В функции триггера используются следующие специальные переменные:

- NEW — содержит значения полей создаваемой записи задачи (employee_id, project_id, budget_required, priority и другие). Эти значения используются для проверки прав сотрудника, доступности бюджета и соответствия лимитам.
- TG_OP — имя операции, используется для формирования сообщения в журнале аудита.

- TG_TABLE_NAME — имя таблицы, в которую вставляется запись, используется для контекстного логирования.

Если какое-либо из проверок не проходит, функция прерывает операцию командой RAISE EXCEPTION с сообщением об ошибке, которое будет передано приложению и позволит пользователю понять причину отказа.

8.2. Описание затрагиваемых таблиц и их назначения

Для реализации триггера в схему добавлены следующие отношения, описывающие предметную область системы управления задачами.

https://github.com/MKoblents/data_base/blob/main/db_lab_3/table_emp.sql

Таблица Employee

Атрибуты: employee_id, full_name, position, department, max_active_tasks, can_manage_budget.

Первичный ключ: employee_id.

Назначение: хранит данные о сотрудниках, включая лимит на количество одновременных задач и полномочия по управлению бюджетом.

Таблица Project

Атрибуты: project_id, project_name, description, total_budget, spent_budget, status, created_at.

Первичный ключ: project_id.

Назначение: фиксирует информацию о проектах, включая общий и израсходованный бюджет, текущий статус.

Таблица Task

Атрибуты: task_id, title, description, employee_id, project_id, priority, budget_required, status, created_at, deadline.

Первичный ключ: task_id.

Назначение: хранит задачи, назначенные сотрудникам в рамках проектов, с указанием приоритета, требуемого бюджета и сроков.

Таблица Employee_Project_Rights

Атрибуты: right_id, employee_id, project_id, can_assign_tasks, can_edit_budget, max_tasks_in_project.

Первичный ключ: right_id.

Уникальный ключ: {employee_id, project_id}.

Назначение: определяет права сотрудников на выполнение действий в конкретных проектах.

Таблица Notification

Атрибуты: notification_id, employee_id, message, created_at, is_read, notification_type.

Первичный ключ: notification_id.

Назначение: хранит уведомления для сотрудников о назначенных задачах, изменениях статуса или нарушениях правил.

Таблица Task_Audit

Атрибуты: audit_id, task_id, operation_type, check_result, message, checked_at, checked_by.

Первичный ключ: audit_id.

Назначение: ведет журнал проверок, выполненных триггером при создании задач, что обеспечивает аудируемость и отладку бизнес-логики.

Таблица Budget_Reservation

Атрибуты: reservation_id, task_id, project_id, reserved_amount, reserved_at, status, committed_at.

Первичный ключ: reservation_id.

Назначение: фиксирует временное резервирование бюджета под конкретную задачу до момента ее выполнения, завершения или отмены.

8.3. Код создания схемы базы данных

https://github.com/MKoblents/data_base/blob/main/db_lab_3/f.sql

```
SET search_path TO task_manager;
```

```
CREATE TABLE Employee (  
    employee_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(255) NOT NULL,  
    position VARCHAR(100) NOT NULL,  
    department VARCHAR(100),  
    max_active_tasks INTEGER NOT NULL DEFAULT 5 CHECK (max_active_tasks >= 0),  
);
```

```
CREATE TABLE Project (  
    project_id SERIAL PRIMARY KEY,  
    project_name VARCHAR(255) NOT NULL,  
    description TEXT,  
    total_budget NUMERIC(12, 2) NOT NULL CHECK (total_budget >= 0),  
    spent_budget NUMERIC(12, 2) NOT NULL DEFAULT 0 CHECK (spent_budget >= 0),
```

```

        status VARCHAR(50) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE',
'ON_HOLD', 'COMPLETED', 'CANCELLED')),
        created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
        CHECK (spent_budget <= total_budget)
    );

```

```

CREATE TABLE Task (
    task_id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    employee_id INTEGER NOT NULL REFERENCES Employee(employee_id) ON
DELETE CASCADE,
    project_id INTEGER NOT NULL REFERENCES Project(project_id) ON DELETE
CASCADE,
    priority VARCHAR(20) NOT NULL CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH',
'CRITICAL')),
    budget_required NUMERIC(10, 2) DEFAULT 0 CHECK (budget_required >= 0),
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING' CHECK (status IN
('PENDING', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED')),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    deadline DATE
);

```

```

CREATE TABLE Employee_Project_Rights (
    right_id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES Employee(employee_id) ON
DELETE CASCADE,
    project_id INTEGER NOT NULL REFERENCES Project(project_id) ON DELETE
CASCADE,
    can_assign_tasks BOOLEAN NOT NULL DEFAULT FALSE,
    can_edit_budget BOOLEAN NOT NULL DEFAULT FALSE,
    max_tasks_in_project INTEGER NOT NULL DEFAULT 3 CHECK
(max_tasks_in_project >= 0),
    UNIQUE(employee_id, project_id)
);

```

```

CREATE TABLE Notification (
    notification_id SERIAL PRIMARY KEY,
    employee_id INTEGER NOT NULL REFERENCES Employee(employee_id) ON
DELETE CASCADE,
    message TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_read BOOLEAN NOT NULL DEFAULT FALSE,
    notification_type VARCHAR(50) NOT NULL DEFAULT 'TASK_ASSIGNMENT'
);

```

);

```
CREATE TABLE Task_Audit (  
    audit_id SERIAL PRIMARY KEY,  
    task_id INTEGER,  
    operation_type VARCHAR(20) NOT NULL,  
    check_result VARCHAR(20) NOT NULL CHECK (check_result IN ('PASS', 'FAIL')),  
    message TEXT,  
    checked_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    checked_by VARCHAR(100) DEFAULT 'SYSTEM_TRIGGER'  
);
```

```
CREATE TABLE Budget_Reservation (  
    reservation_id SERIAL PRIMARY KEY,  
    task_id INTEGER NOT NULL,  
    project_id INTEGER NOT NULL REFERENCES Project(project_id) ON DELETE  
CASCADE,  
    reserved_amount NUMERIC(10, 2) NOT NULL CHECK (reserved_amount > 0),  
    reserved_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE',  
'COMMITTED', 'CANCELLED')),  
    committed_at TIMESTAMP  
);
```

8.4. Реализация триггерной функции

```
CREATE OR REPLACE FUNCTION check_task_assignment_rules()  
RETURNS TRIGGER AS $$  
DECLARE  
    v_conn TEXT := 'dbname=' || current_database();  
    v_audit_sql TEXT;  
    v_has_rights BOOLEAN;  
    v_active_tasks INTEGER;  
    v_max_tasks INTEGER;  
    v_available_budget NUMERIC(12, 2);  
    v_can_manage_budget BOOLEAN;  
    v_employee_name VARCHAR(255);  
    v_project_name VARCHAR(255);  
    v_project_status VARCHAR(50);  
BEGIN  
    SELECT full_name INTO v_employee_name FROM task_manager.employee WHERE  
employee_id = NEW.employee_id;  
    SELECT project_name, status INTO v_project_name, v_project_status FROM  
task_manager.project WHERE project_id = NEW.project_id;
```

```

v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id, operation_type,
check_result, message) VALUES (%s, %L, %L, %L)',
NEW.task_id, 'INSERT', 'PENDING', 'Начата проверка условий
назначения задачи');
PERFORM dblink_exec(v_conn, v_audit_sql);

IF v_project_status != 'ACTIVE' THEN
v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id, operation_type,
check_result, message) VALUES (%s, %L, %L, %L)',
NEW.task_id, 'INSERT', 'FAIL', 'Проект "' || v_project_name || '" не
находится в активном статусе');
PERFORM dblink_exec(v_conn, v_audit_sql);
RAISE EXCEPTION 'Ошибка назначения задачи: проект "%" не активен.',
v_project_name;
END IF;

SELECT can_assign_tasks, max_tasks_in_project, can_edit_budget
INTO v_has_rights, v_max_tasks, v_can_manage_budget
FROM task_manager.employee_project_rights
WHERE employee_id = NEW.employee_id AND project_id = NEW.project_id;

IF NOT FOUND OR NOT v_has_rights THEN
v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id, operation_type,
check_result, message) VALUES (%s, %L, %L, %L)',
NEW.task_id, 'INSERT', 'FAIL', 'Сотрудник не имеет прав на
выполнение задач в проекте');
PERFORM dblink_exec(v_conn, v_audit_sql);
RAISE EXCEPTION 'Ошибка назначения задачи: сотрудник "%" не уполномочен
выполнять задачи в проекте "%".', v_employee_name, v_project_name;
END IF;

SELECT COUNT(*) INTO v_active_tasks
FROM task_manager.task
WHERE employee_id = NEW.employee_id
AND project_id = NEW.project_id
AND status IN ('PENDING', 'IN_PROGRESS');

IF v_active_tasks >= v_max_tasks THEN
v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id, operation_type,
check_result, message) VALUES (%s, %L, %L, %L)',
NEW.task_id, 'INSERT', 'FAIL', 'Превышен лимит активных задач');
PERFORM dblink_exec(v_conn, v_audit_sql);

```



```

        RAISE EXCEPTION 'Ошибка назначения задачи: сотрудник "%" уже выполняет
максимальное количество задач (%) в проекте "%".', v_employee_name, v_max_tasks,
v_project_name;
    END IF;

```

```

    IF NEW.budget_required > 0 THEN
        SELECT COALESCE(total_budget - spent_budget, 0) INTO v_available_budget
        FROM task_manager.project WHERE project_id = NEW.project_id;

```

```

        IF v_available_budget < NEW.budget_required THEN
            v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id,
operation_type, check_result, message) VALUES (%s, %L, %L, %L)',
                NEW.task_id, 'INSERT', 'FAIL', 'Недостаточно бюджета на проекте');
            PERFORM dblink_exec(v_conn, v_audit_sql);
            RAISE EXCEPTION 'Ошибка назначения задачи: в проекте "%" недостаточно
бюджета (доступно: %, требуется: %).', v_project_name, v_available_budget,
NEW.budget_required;
        END IF;

```

```

        IF NOT v_can_manage_budget AND NEW.budget_required > 100.00 THEN
            v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id,
operation_type, check_result, message) VALUES (%s, %L, %L, %L)',
                NEW.task_id, 'INSERT', 'FAIL', 'Сотрудник не уполномочен
расходовать бюджет свыше установленного порога');
            PERFORM dblink_exec(v_conn, v_audit_sql);
            RAISE EXCEPTION 'Ошибка назначения задачи: сотрудник "%" не имеет
полномочий на расходование бюджета в размере %. ', v_employee_name,
NEW.budget_required;
        END IF;

```

```

        INSERT INTO task_manager.budget_reservation (task_id, project_id, reserved_amount,
status)
        VALUES (NEW.task_id, NEW.project_id, NEW.budget_required, 'ACTIVE');

```

```

        UPDATE task_manager.project SET spent_budget = spent_budget +
NEW.budget_required WHERE project_id = NEW.project_id;
    END IF;

```

```

    INSERT INTO task_manager.notification (employee_id, message, notification_type)
    VALUES (NEW.employee_id, 'Вам назначена задача: "' || NEW.title || '" в проекте "' ||
v_project_name || "'.', 'TASK_ASSIGNMENT');

```

```

    v_audit_sql := format('INSERT INTO task_manager.task_audit (task_id, operation_type,
check_result, message) VALUES (%s, %L, %L, %L)',

```

```

NEW.task_id, 'INSERT', 'PASS', 'Задача успешно назначена. Резерв
бюджета: ' || COALESCE(NEW.budget_required, 0));
PERFORM dblink_exec(v_conn, v_audit_sql);

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

8.5. Создание триггера

```

CREATE TRIGGER trg_check_task_assignment
BEFORE INSERT ON Task
FOR EACH ROW
EXECUTE FUNCTION check_task_assignment_rules();

```

8.6. Примеры демонстрации работы триггера

Пример 1. Корректная операция — задача назначена успешно
Условие: сотрудник с id = 1 имеет права на проект с id = 1, лимит задач не исчерпан, бюджет достаточен.

```

-- Предварительная подготовка данных
INSERT INTO Employee (full_name, position, max_active_tasks) VALUES ('Анна
Петрова', 'Разработчик', 5);
INSERT INTO Project (project_name, total_budget, spent_budget) VALUES ('Разработка
модуля аналитики', 5000.00, 1200.00);
INSERT INTO Employee_Project_Rights (employee_id, project_id, can_assign_tasks,
can_edit_budget, max_tasks_in_project) VALUES (1, 1, TRUE, TRUE, 3);

-- Назначение задачи
INSERT INTO Task (title, description, employee_id, project_id, priority, budget_required,
deadline)
VALUES ('Реализовать экспорт отчетов', 'Добавить возможность экспорта в PDF и
Excel', 1, 1, 'HIGH', 150.00, '2026-06-01');

```

Ожидаемый результат: запись успешно добавлена в Task, создана запись в Notification, зарезервирован бюджет в Budget_Reservation, обновлен spent_budget в Project, в Task_Audit зафиксирован результат 'PASS'.

```
planet_db=# INSERT INTO Task (title, description, employee_id, project_id, priority, budget_required, deadline)
VALUES ('Реализовать экспорт отчётов', 'Добавить возможность экспорта в PDF и Excel', 1, 1, 'HIGH', 150.00, '2026-06-01');
INSERT 0 1
planet_db=# select * from task_audit
;
```

audit_id	task_id	operation_type	check_result	message	checked_at	checked_by
3	3	INSERT	PASS	Задача успешно назначена. Резерв бюджета: 150.00	2026-05-04 16:27:27.31433	SYSTEM_TRIGGER

```
(1 row)

planet_db=# select * from task;
task_id | title | description | employee_id | project_id | priority | budget_required | status | created_at | deadline
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
3 | Реализовать экспорт отчётов | Добавить возможность экспорта в PDF и Excel | 1 | 1 | HIGH | 150.00 | PENDING | 2026-05-04 16:27:27.31433 | 2026-06-01
(1 row)
```

Пример 2. Некорректная операция — отсутствие прав у сотрудника

Условие: сотрудник с id = 2 не имеет записи в Employee_Project_Rights для проекта с id = 1.

-- Предварительная подготовка: создаем сотрудника без прав на проект

```
INSERT INTO Employee (full_name, position, max_active_tasks) VALUES ('Иван Сидоров', 'Тестировщик', 4);
```

-- Попытка назначения задачи

```
INSERT INTO Task (title, employee_id, project_id, priority, budget_required)
VALUES ('Протестировать модуль', 2, 1, 'MEDIUM', 50.00);
```

Ожидаемый результат: операция прервана с сообщением об ошибке: «Ошибка назначения задачи: сотрудник "Иван Сидоров" не уполномочен выполнять задачи в проекте "Разработка модуля аналитики"». В Task_Audit записан результат 'FAIL' с соответствующим сообщением. Запись в Task не создана.

```
planet_db=# INSERT INTO Task (title, employee_id, project_id, priority, budget_required)
VALUES ('Протестировать модуль', 2, 1, 'MEDIUM', 50.00);
ERROR: Ошибка назначения задачи: сотрудник "Иван Сидоров" не уполномочен выполнять задачи в проекте "Разработка модуля аналитики".
CONTEXT: PL/pgSQL function check_task_assignment_rules() line 37 at RAISE
planet_db=#
```

```
planet_db=# select * from task_audit
planet_db=# ;
```

audit_id	task_id	operation_type	check_result	message	checked_at	checked_by
3	3	INSERT	PASS	Задача успешно назначена. Резерв бюджета: 150.00	2026-05-04 16:27:27.31433	SYSTEM_TRIGGER
6	10	INSERT	PENDING	Начата проверка условий назначения задачи	2026-05-04 17:13:18.657866	SYSTEM_TRIGGER
7	10	INSERT	FAIL	Сотрудник не имеет прав на выполнение задач в проекте	2026-05-04 17:13:18.669955	SYSTEM_TRIGGER
8	11	INSERT	PENDING	Начата проверка условий назначения задачи	2026-05-05 10:50:27.025699	SYSTEM_TRIGGER
9	11	INSERT	FAIL	Сотрудник не имеет прав на выполнение задач в проекте	2026-05-05 10:50:27.03596	SYSTEM_TRIGGER

```
(5 rows)
```

Пример 3. Пограничный случай — задача без расхода бюджета

Условие: назначается задача «Провести код-ревью» (не требует значительных расходов), сотрудник имеет права, но не имеет полномочий на управление бюджетом.

-- Предварительная подготовка: сотрудник без прав на бюджет

```
INSERT INTO Employee (full_name, position, max_active_tasks) VALUES ('Мария Козлова', 'Аналитик', 3);
```

```
INSERT INTO Employee_Project_Rights (employee_id, project_id, can_assign_tasks,
max_tasks_in_project, can_edit_budget) VALUES (3, 1, TRUE, 2, FALSE);
```

-- Назначение задачи с минимальным бюджетом

```
INSERT INTO Task (title, employee_id, project_id, priority, budget_required)
```

```
VALUES ('Провести код-ревью', 3, 1, 'LOW', 20.00);
```

Ожидаемый результат: операция выполнена успешно, так как требуемый бюджет (20.00) не превышает порог для сотрудников без полномочий (100.00). Уведомление создано, аудит зафиксировал успех.

```
planet_db=# INSERT INTO Task (title, employee_id, project_id, priority, budget_required)
VALUES ('Провести код-ревью', 3, 1, 'LOW', 20.00);
INSERT 0 1
```